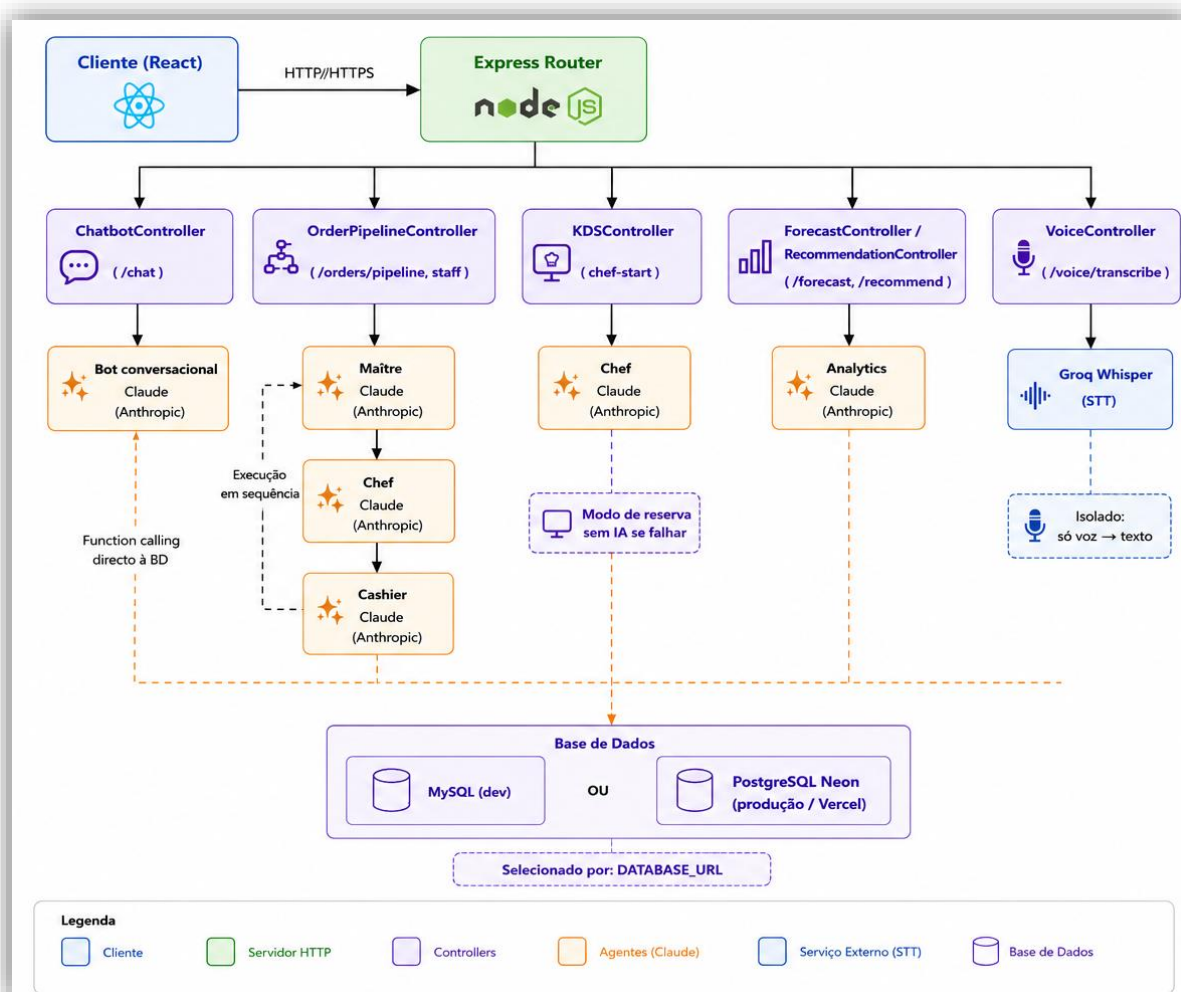


Relatório Final - SmartBistro AI



Nome do aluno:

DANILSON JORGE TAVARES SANCHES

Número:

UPSKILL217

Índice

1. Introdução	3
2. Auditoria de Código	4
3. Segurança — autenticação e autorização	5
4. Base de dados de produção (Neon)	6
5. Deploy e configuração	7
6. Correção de bugs no chatbot	8
7. Resiliência a falhas de IA	10
8. A Aplicação	11
9. Conclusão	17

1. Introdução

Este documento descreve o trabalho realizado na fase final de preparação do projeto — auditoria de código, correções de segurança, migração da base de dados de produção, correção de bugs no chatbot e reconstrução da apresentação com exemplos visuais da aplicação em funcionamento.

2. Auditoria de Código

Foi feita uma auditoria completa ao backend e frontend, identificando ~50 pontos de melhoria organizados por impacto. Dos itens de risco mais baixo, foram corrigidos nesta fase:

- **Código morto removido:** dois ficheiros de chat nunca usados (ChatBot.jsx, Chat.jsx), constantes de cor não referenciadas (BAR_STATUS_COLORS, STATUS_LABELS), o ficheiro models.js do backend (o modelo Claude já vem do .env).
- **Formatação de moeda consolidada:** existiam 4 funções diferentes a formatar euros de forma inconsistente (€4.00 vs 4,00 €); unificado num único fmtEur em formato pt-PT.
- **ESLint:** corrigidos os 10 erros seguros e reais (variáveis não usadas, um bug no comparador de sort dos alertas de stock, um catch vazio a esconder falhas de streaming SSE). Os ~45 avisos restantes são todos do mesmo padrão de polling manual, cuja correcção correcta é uma migração para React Query — adiada por ser uma mudança maior, demasiado arriscada tão perto da apresentação.
- **Comentários:** adicionados a todas as funções, classes e constantes exportadas nos utils e em vários componentes de UI que não tinham nenhum.

3. Segurança — autenticação e autorização

A auditoria revelou que **quase todas as rotas do backend estavam completamente abertas**, sem qualquer verificação de token. A correcção foi feita em duas fases, para não quebrar o site:

1. **Injecção automática do JWT no frontend** (api.js / BaseService.js) — até aqui, apenas 4 dos ~18 services enviavam manualmente o header Authorization; os restantes (pedidos, mesas, utilizadores, stock, facturas, pagamentos...) nunca enviavam o token. Corrigido para que todos os pedidos HTTP anexem automaticamente o JWT guardado em cookie.
2. **Protecção das rotas** com os middlewares verifyToken / requireRole, rota a rota, distinguindo três grupos:
 - **Só staff** (mesas, stock, clientes, ingredientes, reservas, logs...)
 - **Staff + cliente autenticado** (pedidos, facturas, pagamentos, notificações)
 - **Público** (menu do cardápio, login/registo, chat — incluindo convidados anónimos)

Durante este processo foram detectados e corrigidos dois conflitos reais: o widget de chat (usado por visitantes anónimos) chama tableService.updateStatus para libertar a mesa após pagamento, e chatService.getConversations()/getChatHistory() para recarregar o histórico — ambos tiveram de ficar deliberadamente públicos, apesar de tecnicamente “abertos”, porque servem pedidos genuínos de convidados sem conta.

4. Base de dados de produção (Neon)

A aplicação usa MySQL em desenvolvimento local e PostgreSQL (Neon) em produção, através da mesma camada de acesso (db.js, selecção automática por DATABASE_URL). Uma verificação directa ao schema da base de dados **ao vivo** no Neon revelou que estava desactualizada face ao ambiente local:

Em falta na Neon	Funcionalidade afectada
Tabela table_groups	Juntar mesas
Tabela table_group_members	Juntar mesas
Coluna orders.group_id	Juntar mesas
Tabela weekly_forecast	Previsão de vendas (Relatórios)

Foi aplicada uma migração aditiva (CREATE TABLE IF NOT EXISTS / ADD COLUMN IF NOT EXISTS, dentro de uma transacção com rollback automático) directamente na base de dados de produção — sem apagar nada, sem perda de dados. Confirmado depois que o schema ao vivo ficou 100% alinhado com o local.

5. Deploy e configuração

No Vercel, faltavam as variáveis `ANTHROPIC_API_KEY` e `CLAUDE_MODEL` — só existia `GROQ_API_KEY`, resquício da arquitectura anterior à migração para Claude. Configuradas ambas em produção (sem nunca expor a chave na conversa) e feito um novo deploy, confirmado com testes directos à API (health check e leitura da base de dados).

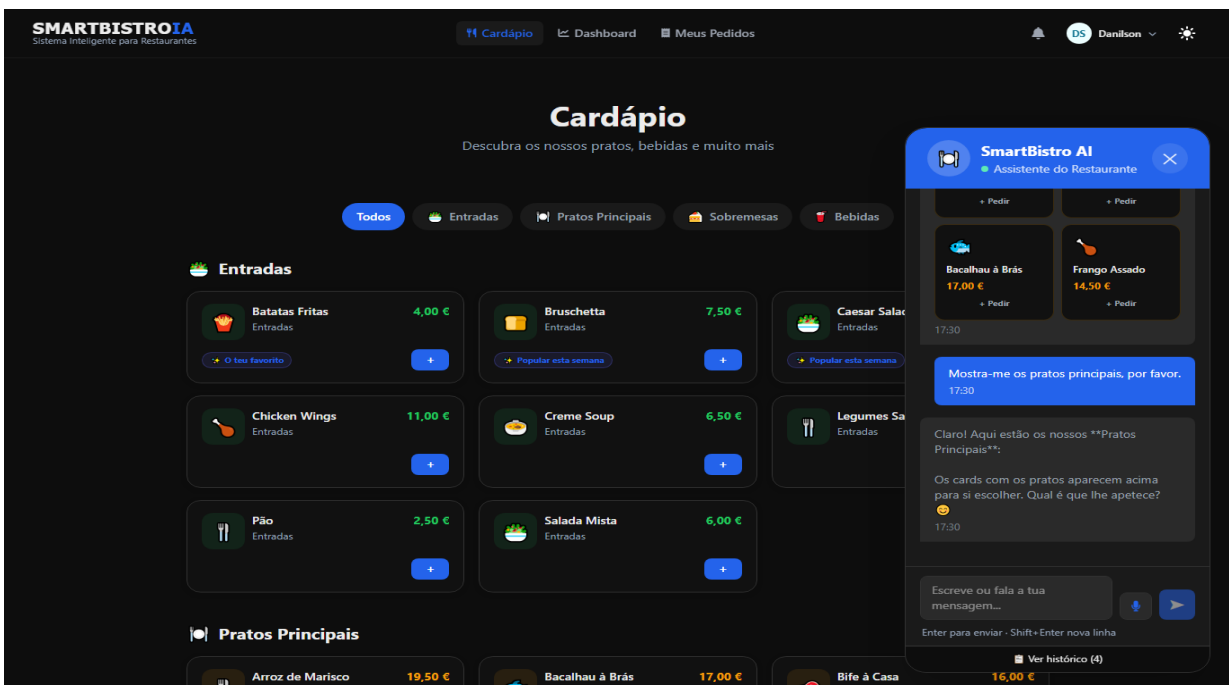
6. Correção de bugs no chatbot

Durante um teste real de conversa, foi identificado que o bot, ao ser confirmado que o cliente queria ver uma categoria do menu (ex: “quero bebidas”), respondia com texto a dizer que ia mostrar a lista — mas sem chamar a função `get_items` que efectivamente mostra os cards ao cliente. Só depois de o cliente insistir várias vezes é que a lista aparecia.

Antes da correção:

Cliente: “quero bebidas” Bot: “Aqui estão as nossas bebidas — qual é a sua escolha?” *(sem mostrar nada)* Cliente: “não vi nada” Bot: *(só agora mostra a lista, depois de insistência)*

No `CHATBOT_SYSTEM_PROMPT`, a instrução passou a exigir explicitamente que a ferramenta `get_items` seja chamada **no mesmo turno**, antes de qualquer texto sobre a categoria, terminando sempre a mensagem com um convite à escolha. Confirmado em teste ao vivo que os cards do menu aparecem de imediato:



Chat a mostrar o cardápio em cards

O mesmo chatbot também confirma reservas em linguagem natural, validando data, número de pessoas e contacto antes de reservar a mesa:

The screenshot displays the SMARTBISTROIA web application. The header includes the logo 'SMARTBISTROIA Sistema Inteligente para Restaurantes' and navigation links for 'Entrar' and 'Registar'. The main section is titled 'Cardápio' (Menu) with the subtitle 'Descubra os nossos pratos, bebidas e muito mais'. Below this are tabs for 'Todos', 'Entradas', 'Pratos Principais', 'Sobremesas', and 'Bebidas'. The 'Entradas' (Starters) section is active, showing items like Batatas Fritas (4,00 €), Bruschetta (7,50 €), Caesar Salad, Chicken Wings (11,00 €), Creme Soup (6,50 €), Pão (2,50 €), and Salada Mista (6,00 €). The 'Pratos Principais' (Main Dishes) section shows Arroz de Marisco (19,50 €), Bacalhau à Brás (17,00 €), and Bife à Casa. A chatbot window titled 'SmartBistro AI - Assistente do Restaurante' is overlaid on the right. It contains a conversation where the user asks to reserve a table for 2 people for tomorrow at 20h. The chatbot responds with the reservation details and confirms the booking.

SMARTBISTROIA
Sistema Inteligente para Restaurantes

[Entrar](#) [Registar](#)

Cardápio

Descubra os nossos pratos, bebidas e muito mais

[Todos](#) [Entradas](#) [Pratos Principais](#) [Sobremesas](#) [Bebidas](#)

Entradas

- Batatas Fritas** Entradas 4,00 €
- Bruschetta** Entradas 7,50 €
- Caesar Salad** Entradas
- Chicken Wings** Entradas 11,00 €
- Creme Soup** Entradas 6,50 €
- Legumes Sa** Entradas
- Pão** Entradas 2,50 €
- Salada Mista** Entradas 6,00 €

Pratos Principais

- Arroz de Marisco** Pratos Principais 19,50 €
- Bacalhau à Brás** Pratos Principais 17,00 €
- Bife à Casa** Pratos Principais

SmartBistro AI
Assistente do Restaurante

Olá, sou a Sofia Martins, o meu número é 912345678. Quero reservar uma mesa para 2 pessoas amanhã às 20h.
17:19

Olá Sofia! 🍷 Ótimo, vou ajudar com a sua reserva.

Deixe-me confirmar os dados:

- **2 pessoas**
- **Amanhã (06/07/2026) às 20h**
- **Telefone: 912345678**

Perfeito! Vou procurar uma mesa disponível para si. Excelente! Encontrei uma mesa perfeita para si — a **mesa T01** com capacidade para 2 pessoas. Vou confirmar a sua reserva agora.
17:19

Escreve ou fala a tua mensagem...

Enter para enviar · Shift+Enter para nova linha

[Ver histórico \(1\)](#)

Chat a confirmar uma reserva

7. Resiliência a falhas de IA

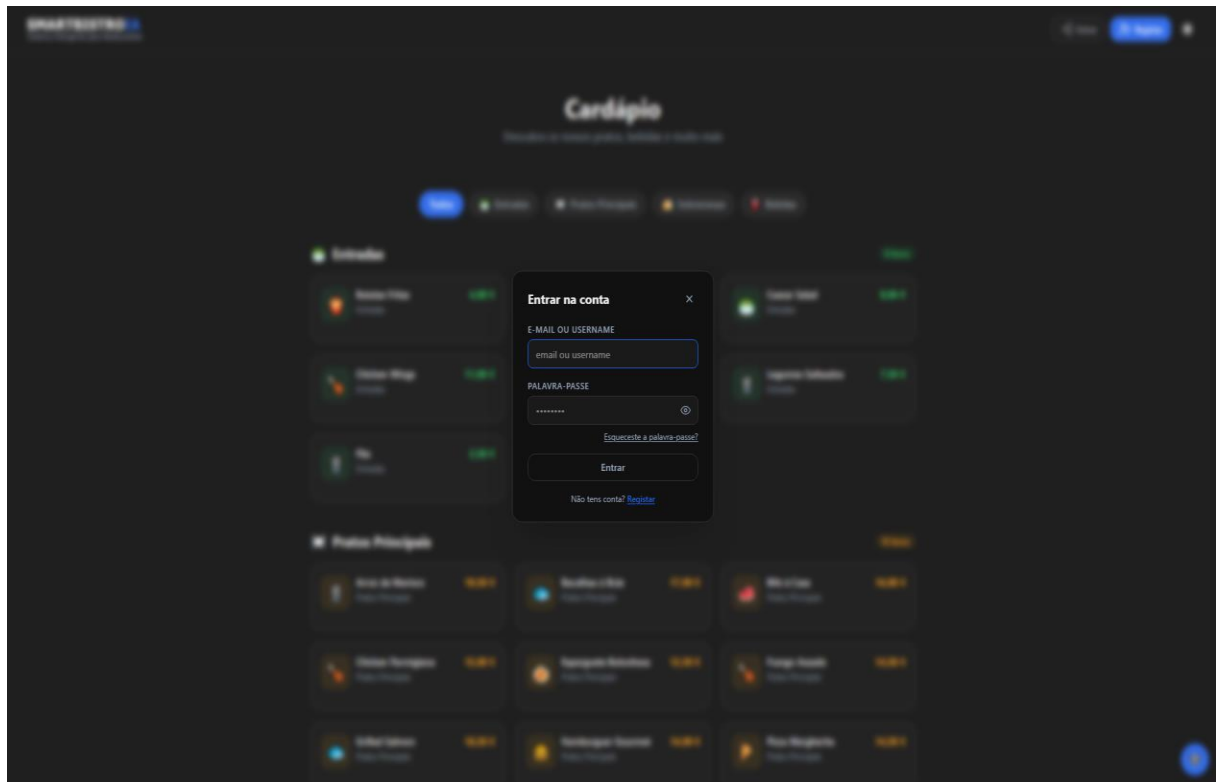
Foi confirmado, ponto a ponto, que **o restaurante nunca fica bloqueado se a IA (Claude) cair**:

- Criar pedidos (cardápio do cliente e modais de staff) — REST puro, sem IA.
- Iniciar preparação no KDS — se o Chef IA falhar, entra automaticamente em **modo de reserva** (sequência genérica + temporizadores, sem IA).
- Mesas, stock, facturas, pagamentos, fechar mesa — cálculo determinístico em JS, nunca pela IA.

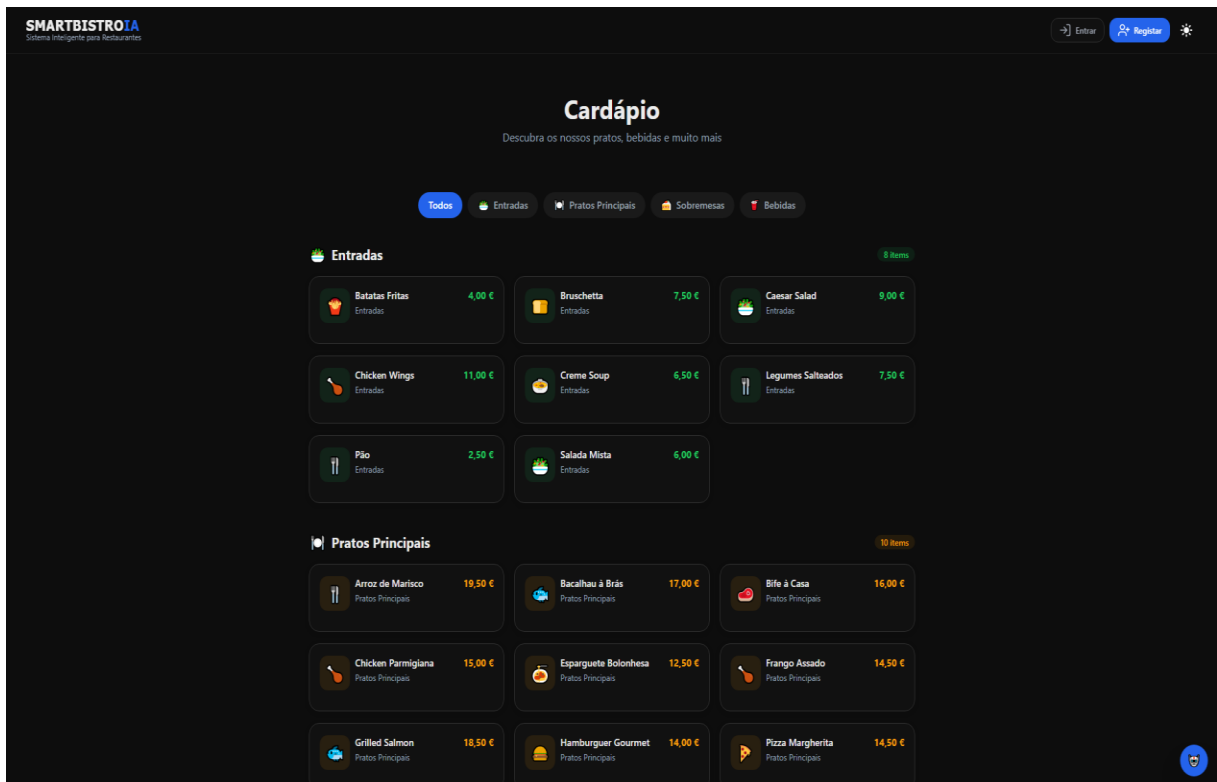
O único que depende inteiramente da IA é, por desenho, a conversa no chat (pedir/reservar por linguagem natural) — mas o cardápio visual e o “fechar mesa” continuam a funcionar em paralelo.

8. A Aplicação

Cliente — cardápio e chat



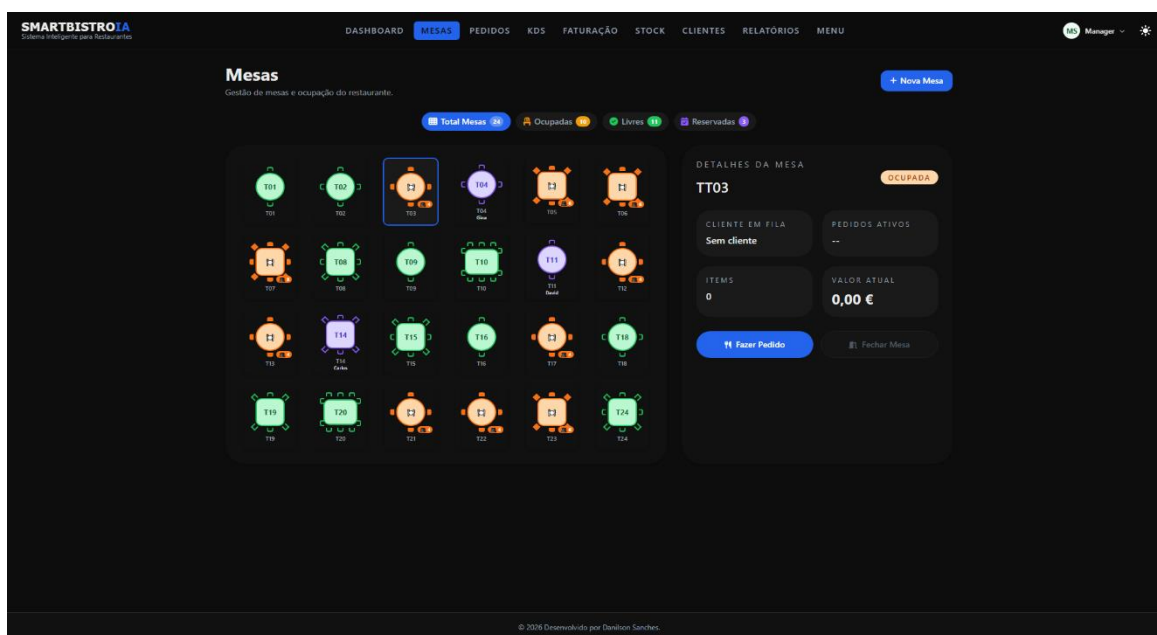
Login



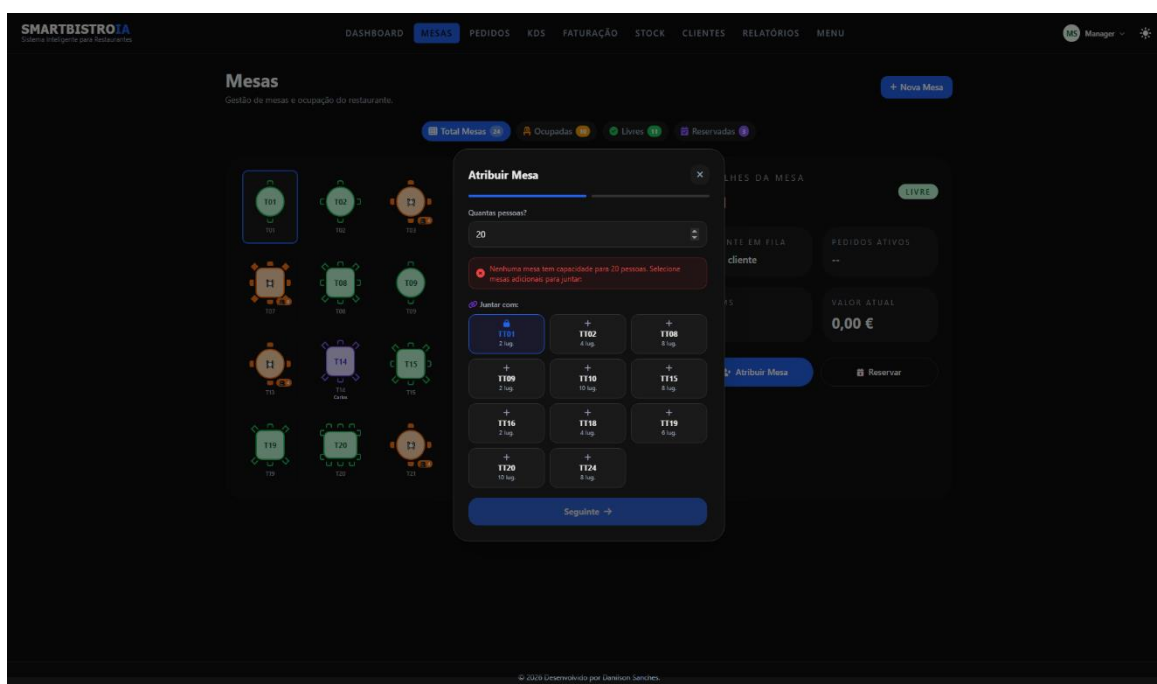
Cardápio

Staff — mesas (atribuir, juntar e reservar)

A funcionalidade mais diferenciadora do projecto: atribuição automática de mesa por capacidade, com opção de juntar várias mesas quando o grupo excede a capacidade de uma só.



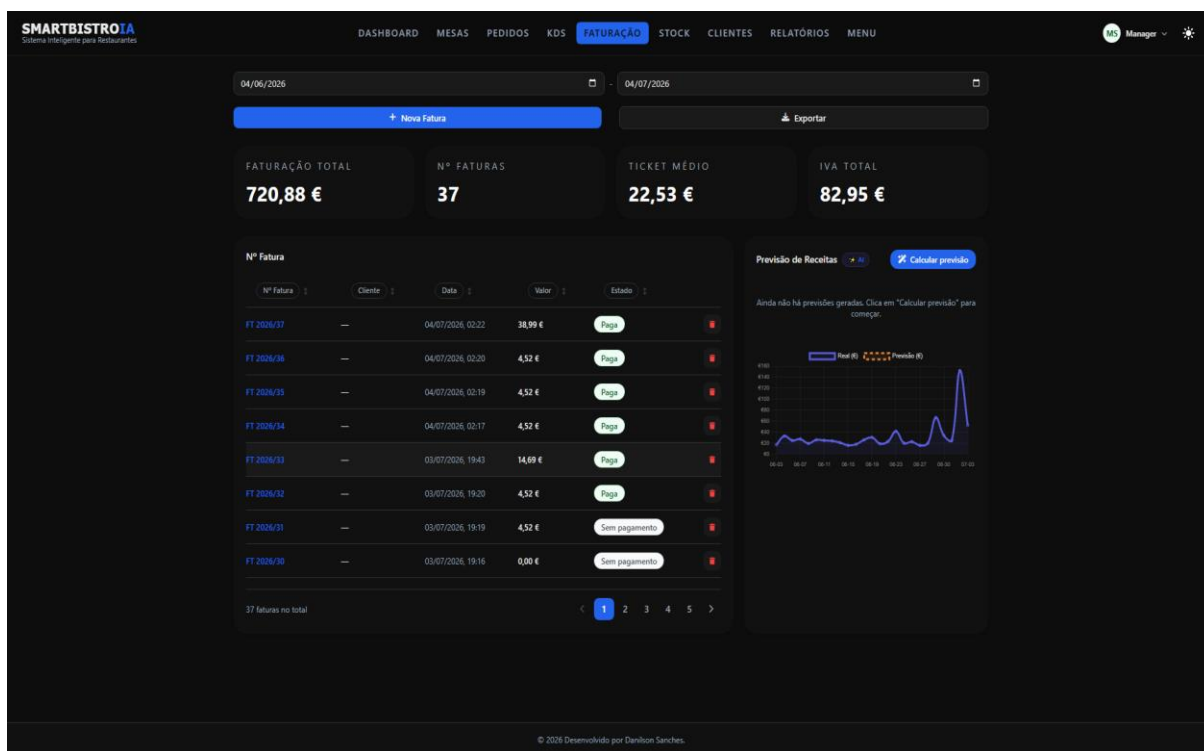
Detalhes da mesa



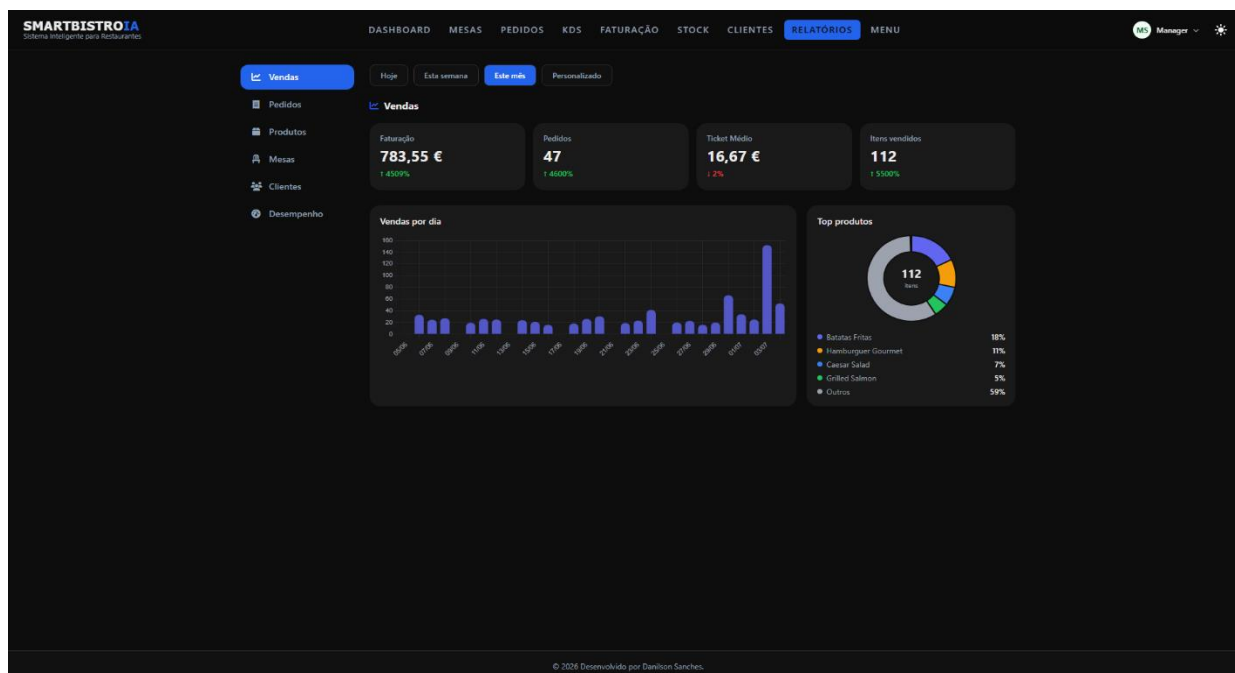
Juntar mesa

KDS — cozinha em tempo real

Gestão do negócio

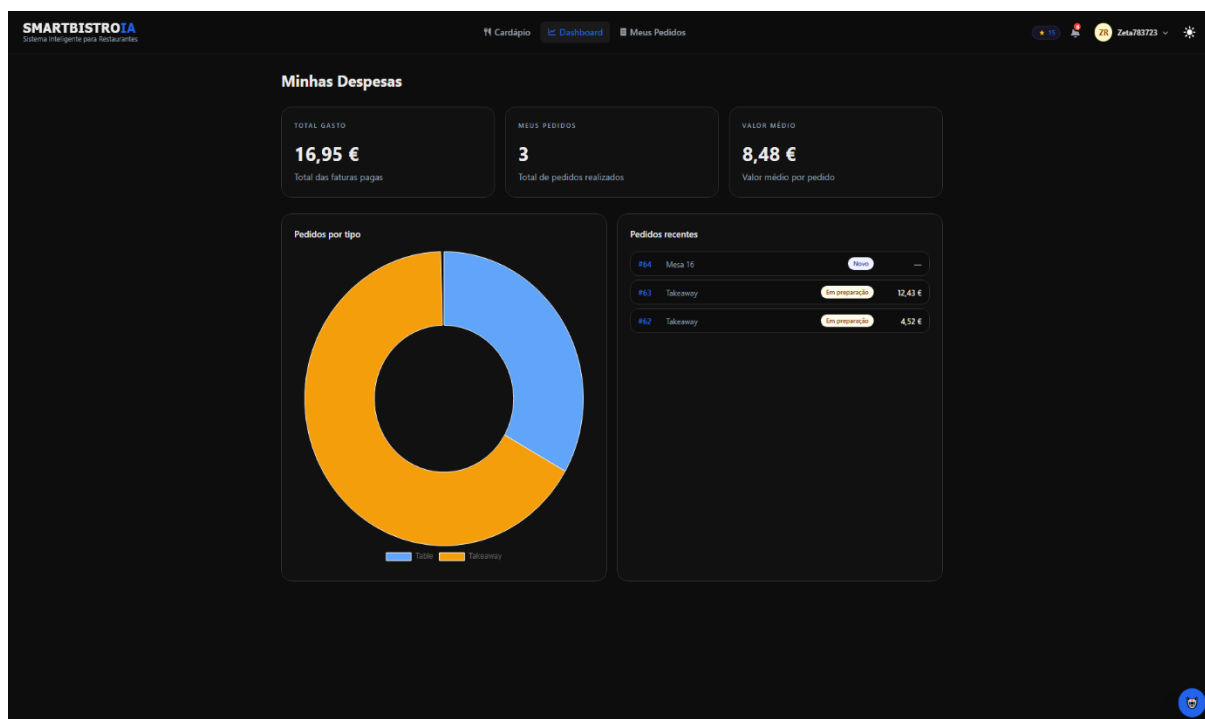


Faturação



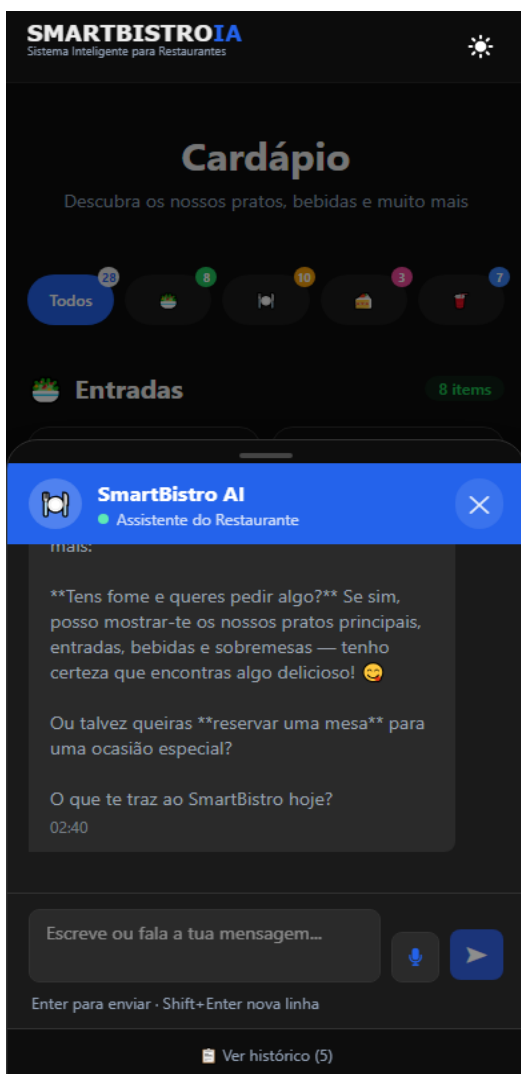
Relatórios

Cliente autenticado — histórico misto (mesa + takeaway)

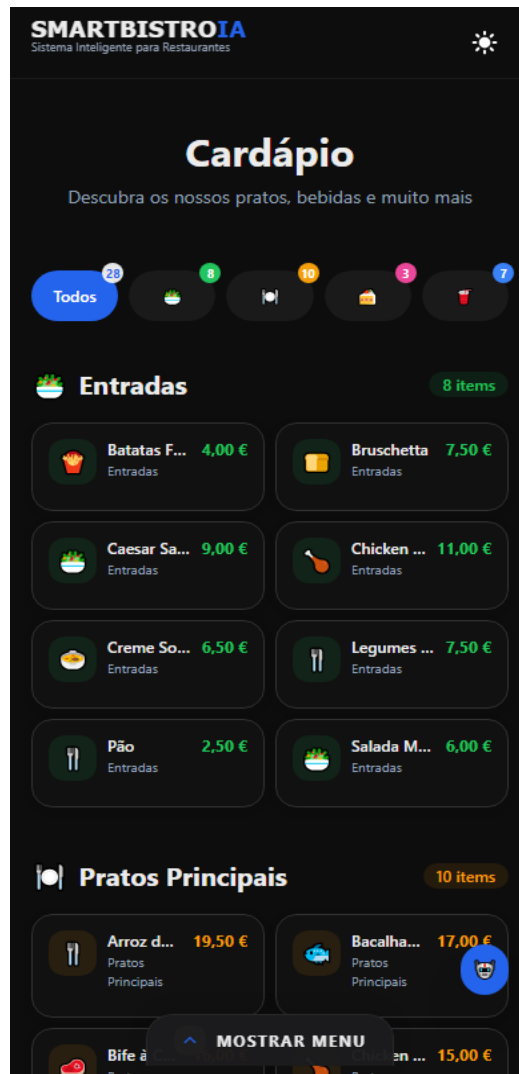


Dashboard do cliente

Responsividade mobile



Cardápio mobile



Chat mobile

9. Conclusão

O trabalho desta fase focou-se em alinhar o código, a base de dados de produção e a apresentação com o estado real e actual do projecto — corrigindo um problema de segurança genuíno (rotas abertas), um bug de comportamento real do chatbot, e uma desactualização silenciosa da base de dados de produção que só foi descoberta por verificação directa. Todas as alterações foram feitas de forma incremental e verificada (testes ao vivo com Playwright, transacções com rollback), dado o prazo apertado até à apresentação.